

# MURU-BENCH: A Benchmark for Mathematical Reasoning Under Uncertainty

Swetank Kumar  
Department of Computer Science  
SRM Institute of Science and Technology (SRMIST)  
Chennai, Tamil Nadu, India  
swetankkumar391@gmail.com  
<https://github.com/swetank18>

## Abstract

Large language models (LLMs) have achieved remarkable performance on mathematical reasoning benchmarks such as GSM8K, MATH, and MMLU. However, these benchmarks exclusively evaluate reasoning toward *single, deterministic correct answers*, neglecting the pervasive role of *genuine mathematical uncertainty* in real-world reasoning tasks. We introduce MURU-BENCH (**M**athematical **R**easoning **U**nder **U**ncertainty **B**enchmark), a dataset of 3,000 problems spanning five categories: Bayesian Updating, Conditional Probability Chains, Distribution Estimation, Decision Under Uncertainty, and Adversarial Ambiguity. Each problem requires a system to (i) identify the correct probabilistic reasoning framework, (ii) execute multi-step mathematical reasoning under parameter uncertainty, and (iii) produce a calibrated answer with a justified confidence interval. Problems are stratified across five difficulty levels via 21 parametric generation templates. Our contribution in this submission is the dataset, the JSON schema, an evaluation harness with six metrics (including Expected Calibration Error and overconfidence rate), an OpenAI/Anthropic/Google API client, and a controlled discriminability study using simulated capability tiers that validates the metric implementations and confirms that the difficulty hierarchy separates known-capability profiles. Evaluations of frontier LLMs are released as ongoing community-driven extensions rather than as headline claims of this paper. Dataset, evaluation code, schema, tests, and reproducibility scaffolding are at <https://github.com/swetank18/MURU>.

## 1 Introduction

The rapid progress of large language models (LLMs) on mathematical benchmarks has been remarkable. Models now achieve near-human performance on GSM8K [1], competitive results on MATH [2], and broad knowledge on MMLU [3]. However, these benchmarks share a fundamental limitation: they test reasoning toward *single, deterministic correct answers*. A model either produces  $3/7$  or it does not; there is no ambiguity in the ground truth.

Real-world mathematical reasoning is rarely so clean. A physician interpreting a diagnostic test must combine uncertain base rates with imperfect test characteristics. An engineer assessing failure risk must propagate uncertainty through a chain of conditional probabilities, some estimated from limited data. A policy-maker must choose actions under uncertain utility functions with unknown state probabilities. In each case, the “correct” answer is not a single number but a *distribution* or *interval*, and the quality of reasoning depends not just on accuracy but on *calibration*—how well the stated confidence matches the actual probability of being correct.

**The Calibrated Uncertainty Gap.** We identify a critical gap in the LLM evaluation landscape: the absence of a mathematical benchmark that requires models to (i) reason formally about probability and uncertainty, (ii) handle ambiguous problem structure where multiple valid formalizations exist, and (iii) produce calibrated confidence in their answers. We term this the **calibrated uncertainty gap**.

This gap matters for AI safety and deployment. An LLM used as a medical reasoning assistant that gives a confident but poorly calibrated probability estimate could lead to harmful decisions. A financial model that ignores parameter uncertainty could understate risk. The current evaluation paradigm—which rewards only accuracy on deterministic problems—provides no signal about these failure modes.

**Our Contributions.** MURU-BENCH addresses this gap with four contributions:

1. **A dataset of 3,000 problems** across five categories of mathematical uncertainty, each with ground-truth point estimates, confidence intervals, step-by-step solutions, and annotated failure modes (Section 3).
2. **An evaluation framework with six metrics**, including Expected Calibration Error (ECE), overconfidence rate, and framework-match rate, that measure not just accuracy but calibration quality and reasoning process (Section 4).
3. **A discriminability study using simulated capability tiers** (Section 5), demonstrating that the metric implementations and difficulty stratification correctly separate model profiles with known accuracy/calibration properties. We are explicit that these are simulator outputs, not measurements of any real model; they validate the harness rather than report capabilities.
4. **An open evaluation harness** for OpenAI, Anthropic, and Google APIs, plus a reproducibility scaffold (deterministic generation, stratified splits, tests, CI), so that frontier-model evaluations can be contributed by the community against a stable benchmark.

We deliberately separate *benchmark release* (the focus of this paper) from *frontier-model evaluation* (in active development; preliminary numbers will be released to the project repository as they are produced). Section 5 therefore reports on simulated tiers only; this lets us validate the harness without making claims about real LLM behavior that we have not yet measured.

## 2 Related Work

**Mathematical Reasoning Benchmarks.** GSM8K [1] tests grade-school arithmetic with deterministic answers and has largely been saturated by frontier models. MATH [2] covers competition-level problems across algebra, geometry, and number theory, but all problems have single correct answers. BIG-Bench Hard [4] includes diverse reasoning tasks but lacks formal probability problems. MathBench [12] evaluates mathematical knowledge across education levels but does not address uncertainty. TheoremQA [13] tests theorem application but with deterministic targets. Notably, none of these benchmarks require producing *calibrated confidence intervals* as part of the answer.

**Calibration and Confidence in LLMs.** Kadavath et al. [5] demonstrated that language models exhibit partial knowledge of their own accuracy, with token probabilities providing a noisy signal of correctness. Guo et al. [6] established Expected Calibration Error (ECE) as the standard metric for neural network calibration. Recent work has explored prompting strategies to elicit calibrated uncertainty [7], showing that verbalized confidence correlates with accuracy but exhibits systematic biases. Xiong et al. [14] studied LLM calibration across diverse tasks and found that larger models tend to be better calibrated but still exhibit substantial miscalibration on novel tasks. Crucially, all of this work treats calibration as a *property of the model’s metacognition*; MURU-BENCH is the first benchmark where calibration is a *first-class evaluation target* with ground-truth calibration intervals.

**Uncertainty Quantification in Machine Learning.** The ML community has extensively studied predictive uncertainty through Bayesian neural networks [8], deep ensembles [9], and conformal prediction [10]. These methods quantify a model’s uncertainty *about its own predictions*. MURU-BENCH tests something fundamentally different: a model’s ability to *reason about uncertainty as a mathematical concept*—computing posterior distributions, propagating parameter uncertainty, and recognizing when a problem admits multiple valid formalizations.

**Adversarial and Ambiguous Evaluation.** TruthfulQA [11] tests whether models produce truthful answers versus popular misconceptions, which relates to our Adversarial Ambiguity category. However, TruthfulQA tests factual knowledge, not mathematical reasoning under structured ambiguity. Our adversarial problems are mathematically

Table 1: MURU-BENCH problem categories. Each category targets a distinct type of mathematical uncertainty, from standard Bayesian inference to deliberately adversarial problem constructions.

| Category                   | Count        | %          | Core Skill Tested                                                         |
|----------------------------|--------------|------------|---------------------------------------------------------------------------|
| Bayesian Updating          | 910          | 30.3       | Sequential prior-to-posterior inference with uncertain likelihoods        |
| Distribution Estimation    | 660          | 22.0       | Estimating population parameters from finite samples with appropriate CIs |
| Decision Under Uncertainty | 525          | 17.5       | Expected utility computation with uncertain state probabilities           |
| Adversarial Ambiguity      | 474          | 15.8       | Recognizing when a problem admits multiple valid formalizations           |
| Cond. Probability Chains   | 431          | 14.4       | Multi-step conditional probability with uncertain intermediates           |
| <b>Total</b>               | <b>3,000</b> | <b>100</b> |                                                                           |

precise: the ambiguity lies in the *formalization* (e.g., Simpson’s Paradox, reference class problems), not in factual uncertainty.

MURU-BENCH is, to our knowledge, the first benchmark that combines formal mathematical reasoning with genuine uncertainty quantification, requiring both correct computation *and* calibrated confidence across a structured difficulty hierarchy.

## 3 Dataset Construction

### 3.1 Design Principles

Each MURU-BENCH problem satisfies five design criteria:

1. **Unambiguous correctness:** A professional mathematician, given the stated assumptions, would agree that the ground-truth answer and confidence interval are correct.
2. **Measurable failure:** An overconfident model would fail in a *quantifiable* way—either its point estimate falls outside the ground-truth CI, or its stated confidence is miscalibrated.
3. **Mathematical uncertainty:** The uncertainty is genuinely mathematical (unknown parameters, ambiguous formalization), not merely linguistic vagueness or world-knowledge uncertainty.
4. **Active reasoning:** The problem cannot be solved by retrieval or pattern matching alone; it requires applying a probabilistic framework to novel numerical inputs.
5. **Non-trivial intervals:** The ground-truth confidence interval is non-degenerate (CI width  $> 0$ ), ensuring that the problem genuinely involves uncertainty.

### 3.2 Problem Categories

We organize problems into five categories, each targeting a distinct type of mathematical uncertainty. Table 1 summarizes the distribution.

**Bayesian Updating** (910 problems). These problems require applying Bayes’ theorem with uncertain priors and/or likelihoods. At lower difficulties, the model must apply standard Bayes’ theorem (e.g., medical diagnostic testing with known sensitivity and specificity). At higher difficulties, problems involve sequential updating with uncertain likelihood ratios or hierarchical models with shrinkage.

**Conditional Probability Chains** (431 problems). Multi-step chains of conditional probabilities where one or more intermediate transition probabilities are uncertain. Templates include sequential pipelines (e.g., multi-stage manufacturing), parallel redundancy systems, and branching cascades with dependent failure modes. The key challenge is correctly propagating uncertainty across multiple conditioning steps.

**Distribution Estimation** (660 problems). Problems requiring inference about population parameters from finite samples. Templates cover sample mean estimation with  $t$ -distribution CIs, Wilson confidence intervals for proportions, variance estimation via  $\chi^2$  distributions, and mixture distribution decomposition. Higher-difficulty problems introduce stratification uncertainty and multi-modal data.

**Decision Under Uncertainty** (525 problems). Expected value and utility computations where state probabilities and/or outcome values are uncertain. Templates include multi-option decisions with sensitivity analysis, sequential decisions with value-of-information calculations, and insurance pricing with Poisson frequency & uncertain severity. These problems test whether models can correctly compose uncertainties in decision-theoretic contexts.

**Adversarial Ambiguity** (474 problems). Problems deliberately designed to exploit overconfident reasoning. Templates include Simpson’s Paradox (where aggregate and subgroup conclusions conflict), reference class problems (where the answer depends on which reference population is used), and anchoring manipulation (where a misleading prior is presented alongside data). These problems have *no single correct point estimate*; instead, the ground-truth is an interval reflecting the range of defensible formalizations.

### 3.3 Parametric Generation

Rather than hand-crafting each problem individually (which would limit scale and introduce stylistic bias), we developed 21 **parametric generation templates**. Each template defines:

- A **mathematical structure** (e.g., “sequential Bayesian updating with  $k$  evidence items and uncertain likelihood ratios”)
- A **parameter space** (e.g., prior  $\in [0.01, 0.5]$ , likelihood ratio  $\in [2, 50]$ )
- A set of **domain contexts** (e.g., medical testing, quality control, forensic evidence) drawn uniformly at random to diversify surface presentation
- A **closed-form solution function** that computes the ground-truth point estimate, confidence interval, and full solution steps given sampled parameters
- **Difficulty modifiers**: additional uncertain parameters or deeper inference chains that increase the difficulty level

Table 9 in Appendix B lists all 21 templates with their categories and difficulty ranges. This approach ensures: (a) numerical diversity within each template (no two instantiations share the same parameters); (b) mathematical correctness by construction (the solution is computed, not estimated); and (c) scalability (new problems can be generated on demand for future benchmark extensions).

### 3.4 Difficulty Calibration

Each problem is assigned a difficulty level from 1 to 5 based on three factors:

The difficulty distribution (Figure 2 in Appendix A) follows a roughly bell-shaped curve centered at D3, with D5 problems deliberately fewer (271, 9.0%) as they require the most sophisticated reasoning and are the most resource-intensive to validate.

### 3.5 Problem Format and Schema

Each problem is stored as a JSON document with the following fields:

- **id**: Unique identifier (e.g., MURU-0001)

Table 2: Difficulty level specification. Each level is defined by the number of uncertain parameters, the depth of the inference chain, and whether adversarial structure is present.

| Level | Description                         | Uncertain Params | Chain Depth | Adversarial? |
|-------|-------------------------------------|------------------|-------------|--------------|
| D1    | Single-formula application          | 0–1              | 1           | No           |
| D2    | Two-step reasoning                  | 1                | 1–2         | No           |
| D3    | Multi-step with one uncertain param | 1–2              | 2–3         | Sometimes    |
| D4    | Compound uncertainty propagation    | 2–3              | 3+          | Sometimes    |
| D5    | Multi-step + adversarial + compound | 3+               | 3+          | Yes          |

- `stem`: Natural-language problem statement with embedded quantitative data
- `category`: One of five categories
- `difficulty`: Integer 1–5
- `uncertainty_type`: Type of uncertainty (e.g., `epistemic_prior`, `parameter_estimation`)
- `required_framework`: The mathematically appropriate framework; one of five values listed in Section 4.1
- `ground_truth`: Object with fields `answer`, `point_estimate`, `confidence_interval`, and `ci_level` (typically 0.95)
- `solution_steps`: Ordered list of reasoning steps
- `common_failure_modes`: List of typical errors a model might make
- `metadata`: Author, review status, source template, and creation timestamp

### 3.6 Quality Assurance Pipeline

All 3,000 problems undergo a three-stage validation pipeline:

**Stage 1: Schema Validation.** Automated checks verify that all required fields are present, types are correct, difficulty is in  $\{1, \dots, 5\}$ , category is from the allowed set, and the confidence interval is well-formed ( $CI_{\text{lower}} < CI_{\text{upper}}$ ).

**Stage 2: Semantic Consistency.** Automated checks verify mathematical consistency: (a) the point estimate falls within the confidence interval; (b) the CI width is non-degenerate and not pathologically large; (c) the CI level matches the problem statement; (d) solution steps reference the correct mathematical operations.

**Stage 3: Spot-Check Review.** Random samples from each (category, difficulty) cell are manually reviewed to verify that the problem stem is clear, the solution is correct, and the failure modes are realistic. Problems that fail any stage are regenerated.

After validation, 3,000 out of 3,000 problems pass all checks (100% pass rate after one round of bug fixes in the Value-of-Information template; see Section D).

### 3.7 Data Splits

We split the dataset into train/validation/test sets using stratified sampling by (category, difficulty) to ensure balanced representation across all 25 cells of the category  $\times$  difficulty matrix:

## 4 Evaluation Methodology

### 4.1 Metrics

We evaluate models on six metrics designed to assess orthogonal dimensions of uncertainty reasoning capability.

Table 3: Data split sizes with stratification by category and difficulty.

| Split      | Count | Purpose                                |
|------------|-------|----------------------------------------|
| Train      | 2,398 | Model fine-tuning (future work)        |
| Validation | 301   | Hyperparameter selection               |
| Test       | 301   | Baseline evaluation (reported results) |

Table 4: MURU-BENCH evaluation metrics. We assess four dimensions: correctness (is the answer right?), calibration (does confidence match accuracy?), process quality (is the reasoning framework correct?), and safety (does the model know what it doesn’t know?).

| Metric          | Dimension   | Range               | Description                            |
|-----------------|-------------|---------------------|----------------------------------------|
| Accuracy@CI     | Correctness | $[0, 1] \uparrow$   | Answer falls within ground-truth CI    |
| ECE             | Calibration | $[0, 1] \downarrow$ | Expected Calibration Error             |
| Overconfidence  | Safety      | $[0, 1] \downarrow$ | Fraction of high-conf wrong answers    |
| Framework Match | Process     | $[0, 1] \uparrow$   | Correct reasoning framework identified |
| Cat. Breakdown  | Diagnostic  | —                   | Per-category accuracy decomposition    |
| Diff. Scaling   | Diagnostic  | —                   | Per-difficulty accuracy decomposition  |

**Accuracy@CI.** A prediction is correct if and only if the model’s point estimate  $\hat{y}$  satisfies  $CI_{\text{lower}} \leq \hat{y} \leq CI_{\text{upper}}$ , where  $[CI_{\text{lower}}, CI_{\text{upper}}]$  is the ground-truth confidence interval. This is strictly harder than exact-match accuracy because the model must produce a *numerically reasonable* estimate, not just the right formula.

**Expected Calibration Error.** We use the standard binned ECE formulation [6]:

$$ECE = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (1)$$

where  $B_m$  is the set of predictions whose stated confidence falls in the  $m$ -th bin,  $\text{acc}(B_m)$  is the accuracy within that bin, and  $\text{conf}(B_m)$  is the mean stated confidence. We use  $M = 10$  equal-width bins. A perfectly calibrated model has  $ECE = 0$ .

**Overconfidence Rate.** The fraction of predictions where the model states confidence  $> 0.7$  but the answer is wrong:  $\text{OvConf} = \frac{|\{i: c_i > 0.7 \wedge \hat{y}_i \notin CI_i\}|}{N}$ . This metric is safety-critical: an overconfident model in a decision-support role could trigger harmful downstream actions.

**Framework Match Rate.** The fraction of predictions where the model identifies the correct mathematical framework (e.g., Bayesian inference vs. frequentist inference). This assesses whether the model understands *why* a particular approach is appropriate, not just whether it can compute the answer.

## 4.2 Prompting Protocol

All models receive a structured system prompt instructing them to:

1. Identify the correct mathematical framework from five options
2. Show step-by-step reasoning
3. Provide a final point estimate as a single number
4. Provide a confidence interval [lower, upper]
5. State a confidence probability between 0 and 1

Table 5: Harness output across five *simulated* responder tiers on the test set ( $n = 301$ ). Tier behavior is *specified* by the simulator (accuracy targets per difficulty plus a confidence distribution), and the table verifies that all six metrics recover the intended ordering. These numbers reflect simulator configuration, not any LLM’s performance.

| Model Tier                     | Acc@CI | ECE↓  | OvConf↓ | FwMatch | $n$ |
|--------------------------------|--------|-------|---------|---------|-----|
| Random Baseline                | 7.3%   | 0.515 | 36.2%   | 33.9%   | 301 |
| Heuristic Baseline             | 31.2%  | 0.470 | 44.5%   | 47.2%   | 301 |
| Competent Model (GPT-3.5 tier) | 49.2%  | 0.239 | 21.6%   | 67.1%   | 301 |
| Strong Model (GPT-4 tier)      | 60.8%  | 0.178 | 20.3%   | 83.7%   | 301 |
| Expert Model (frontier tier)   | 77.1%  | 0.183 | 9.6%    | 89.0%   | 301 |

Responses are parsed via regular expressions to extract four structured fields (FRAMEWORK, POINT\_ESTIMATE, CONFIDENCE\_INTERVAL, CONFIDENCE). Temperature is set to 0 for reproducibility; full prompts are in Appendix E.

### 4.3 Model-Agnostic Evaluation API

The evaluation framework (`evaluation/run_eval.py`) supports three API backends—OpenAI, Anthropic, and Google—with automatic model routing based on the model name prefix. Results are saved as JSON files with full metadata (timestamp, model name, raw responses, parsed predictions, and computed metrics). This enables reproducible evaluation of any model accessible via API.

## 5 Harness Validation via Simulated Capability Tiers

**What this section is, and is not.** This section does *not* report measurements of any language model. We instead validate that the evaluation harness behaves correctly by feeding it five *simulated* responder profiles whose accuracy and calibration properties are specified *a priori*. The purpose is twofold: (1) confirm that the metric implementations (Accuracy@CI, ECE, overconfidence, framework match) are sensitive to capability differences in the expected directions, and (2) confirm that the difficulty stratification (D1–D5) and category structure separate responders along the dimensions they are designed to probe. Real-model evaluations are released to the project repository as they are completed; the present paper does not claim them as findings.

**Simulator construction.** Each tier is a `random.Random` instance parameterized by a per-difficulty success probability and a confidence-distribution shape. The tier names (Random, Heuristic, Competent, Strong, Expert) correspond to qualitative profiles loosely modelled on prior literature reports for non-MURU benchmarks; the specific numerical thresholds are configuration choices, not estimates of any specific model. Source: `evaluation/run_baselines.py, MODEL_PROFILES`.

### 5.1 Discriminability of the Harness

Table 5 reports harness output across all five simulated tiers. Because the simulator profiles were designed to be monotone in accuracy and calibration, *the harness is functioning correctly if and only if it recovers that ordering on every metric*; deviations would indicate metric implementation bugs.

**Harness-validation observations.** All four orderings recovered by the harness are consistent with the simulator’s intended profile, indicating the metric implementations are wired correctly: (1) Accuracy@CI is monotone in tier capability across the full simulator range. (2) ECE decreases as the simulator’s confidence distribution is tightened around its accuracy. The two highest tiers have similar ECE because their confidence distributions are similar by construction; this is an artefact of the simulator design, not a finding about real models. (3) Overconfidence rate decreases monotonically as the simulator’s confidence ceiling tracks accuracy. (4) Framework-match rate is monotone in the per-tier framework-selection probability. None of these constitute claims about LLM behavior.

Table 6: Simulator output by difficulty level. Each per-tier curve reflects the difficulty-specific success probability set in the simulator profile, not measurements of any LLM.

| Model Tier         | D1  | D2  | D3  | D4  | D5  | D1–D5 Gap |
|--------------------|-----|-----|-----|-----|-----|-----------|
| Random Baseline    | 12% | 7%  | 6%  | 10% | 0%  | 12 pp     |
| Heuristic Baseline | 65% | 53% | 18% | 9%  | 0%  | 65 pp     |
| Competent Model    | 81% | 72% | 44% | 22% | 4%  | 77 pp     |
| Strong Model       | 88% | 78% | 63% | 33% | 14% | 74 pp     |
| Expert Model       | 96% | 91% | 81% | 64% | 21% | 75 pp     |

Table 7: Simulator output by problem category. Differences across categories reflect simulator per-category modifiers, not measured model behavior.

| Model Tier         | Adv. Ambiguity | Bayesian Upd. | Cond. Chains | Decision Unc. | Distrib. Est. |
|--------------------|----------------|---------------|--------------|---------------|---------------|
| Random Baseline    | 0%             | 10%           | 0%           | 4%            | 17%           |
| Heuristic Baseline | 4%             | 43%           | 19%          | 23%           | 48%           |
| Competent Model    | 23%            | 61%           | 47%          | 55%           | 48%           |
| Strong Model       | 45%            | 68%           | 47%          | 62%           | 70%           |
| Expert Model       | 49%            | 84%           | 77%          | 74%           | 91%           |

## 5.2 Difficulty Stratification Recovers Configured Profiles

Table 6 and Figure 4 (Appendix A) show simulator output by difficulty level. Because each tier’s per-difficulty accuracy is set as a hyperparameter, the table mainly verifies that the difficulty stratification is faithful: a real LLM’s per-difficulty curve is an empirical question we leave to follow-on evaluations.

The intended interpretation of the difficulty levels is summarized in Table 2: D1–D2 problems are single-formula applications, D3 introduces multi-step reasoning with at least one uncertain parameter, D4 introduces compound uncertainty propagation, and D5 layers adversarial ambiguity onto the D4 conditions. Whether real LLMs exhibit a sharp transition at any specific difficulty level is an empirical question we cannot answer with simulated tiers alone.

## 5.3 Category Decomposition

Table 7 decomposes simulator output by problem category. The simulator applies a small per-category modifier (largest negative on Adversarial Ambiguity), which the harness recovers as expected.

The category ordering recovered by the harness matches the ordering specified in the simulator’s category modifiers. We make no claim that real LLMs would rank categories in this order; that is what frontier-model evaluation will determine.

# 6 Anticipated Failure Modes

## 6.1 Failure Mode Taxonomy

The benchmark is designed around four failure modes that uncertainty-reasoning research has documented in the broader literature. We catalog them here as the diagnostic targets the metrics are designed to detect; the per-category `common_failure_modes` field on every problem records which of these (and template-specific variants) the problem is constructed to elicit. Whether a given LLM exhibits these failures, and at what rates, is an empirical question for downstream evaluation.

**F1: Anchoring on surface statistics.** The responder fixates on the most salient number in the problem stem rather than performing the appropriate probabilistic computation. In Bayesian Updating problems, the canonical instance is reporting the test sensitivity (e.g., 95%) instead of the posterior probability—the well-known base-rate neglect fallacy.



**F2: Ignoring compound uncertainty.** D4–D5 problems require propagating uncertainty through multiple layers (e.g., uncertain prior *and* uncertain likelihood *and* uncertain sample size). A responder that handles each step in isolation but collapses to a point estimate when uncertainties compound will produce a correct-direction answer with a catastrophically narrow confidence interval; the Accuracy@CI metric catches this when the point estimate falls outside the ground-truth interval.

**F3: Single-formalization commitment.** Adversarial Ambiguity problems admit multiple defensible formalizations (e.g., Simpson’s Paradox problems where aggregate and subgroup analyses point opposite directions). A responder that commits to a single formalization with high stated confidence will be penalized by both the Accuracy@CI metric (whose ground-truth interval spans the defensible range) and the overconfidence rate (which fires on high-confidence wrong answers).

**F4: Framework confusion.** Applying frequentist methods where Bayesian methods are appropriate (or vice versa). The framework-match metric is designed to surface this failure mode independently of whether the numerical answer happens to be in range.

## 6.2 Calibration Metric Behavior on Simulated Inputs

Table 8 shows the harness’s calibration metrics on simulator output. Two harness-level observations: (1) ECE responds monotonically to the simulator’s confidence-distribution width but saturates when two tiers share similar confidence shapes, which is the expected behavior of binned ECE under low bin populations; and (2) the overconfidence-rate metric correctly flags the heuristic tier — whose simulator profile applies high confidence to easy-pattern responses — as having the highest rate, illustrating that the metric distinguishes calibration shape from accuracy magnitude. Figure 5 in Appendix A visualizes these patterns. Whether real LLMs exhibit similar plateaus is an empirical question for follow-on work.

Table 8: Calibration metrics from the simulator (not real LLMs).

| Tier (simulated) | ECE↓  | OvConf↓ |
|------------------|-------|---------|
| Random           | 0.515 | 36.2%   |
| Heuristic        | 0.470 | 44.5%   |
| Competent        | 0.239 | 21.6%   |
| Strong           | 0.178 | 20.3%   |
| Expert           | 0.183 | 9.6%    |

## 6.3 Statistical Power of the Test Set

A good benchmark must be able to detect meaningfully different capability profiles. The standard error of an accuracy estimate at  $n = 301$  is approximately  $\sqrt{p(1-p)/n} \approx 2.9$  percentage points at  $p = 0.5$ , so the test set can statistically distinguish two responders whose true accuracies differ by  $\geq 8$  percentage points (two-sided  $\alpha = 0.05$ ). All five simulated-tier inter-tier gaps comfortably exceed this threshold, confirming that the test set is large enough to detect tier-scale capability differences. Tighter discrimination (e.g., between two frontier models within a few percentage points of each other) will require either the validation set, the train set, or paired comparisons via McNemar’s test.

## 7 Discussion

**What this benchmark is for.** MURU-BENCH is intended as a measuring stick for calibrated mathematical reasoning. The dataset and harness alone do not establish that any particular model is or is not well-calibrated; they make such claims *measurable* once frontier-model evaluations are run. The deliberate choice in this paper to keep simulated-tier results separate from any real-model claim is intended to maintain that measurement integrity.

**Why the overconfidence metric matters.** The overconfidence rate is constructed to surface a specific safety-relevant failure: a responder that is high-confidence and wrong. In decision-support contexts (medical, financial, legal), this failure mode is more harmful than uncertainty acknowledged. We conjecture that current frontier LLMs exhibit non-trivial overconfidence rates on D4–D5 problems, but this conjecture requires real-model evaluation to test.

**Why adversarial ambiguity matters.** The Adversarial Ambiguity category targets meta-reasoning: the ability to recognize that a problem admits multiple defensible formalizations. A responder that commits to one formalization with high stated confidence will be penalized by both Accuracy@CI (whose ground truth is an interval over defensible answers) and overconfidence rate. Whether real LLMs systematically fail this category, and to what degree, is the empirical question we hope frontier-model evaluations will answer.

## 8 Limitations

- **Simulated baselines:** The current results use simulated model tiers rather than actual LLM APIs. While this validates MURU-BENCH’s discriminability and difficulty scaling, real model evaluations may reveal different patterns. We release the evaluation framework to enable community evaluation of real models.
- **Parametric generation:** All problems are generated from 21 templates, which may introduce structural patterns that models could exploit. We mitigate this by using diverse domain contexts and numerical randomization, but template-specific overfitting remains possible with sufficient exposure.
- **English only:** All problems are in English, limiting cross-lingual evaluation.
- **Category scope:** While five categories cover major types of mathematical uncertainty, important domains such as stochastic processes, information-theoretic uncertainty, and measure-theoretic probability are not represented.
- **Ground truth subjectivity:** For Adversarial Ambiguity problems, the “correct” answer depends on which formalization is adopted. We address this by providing answer *ranges* rather than point estimates, but the range boundaries themselves involve judgment.
- **Single evaluator:** The quality assurance process relies primarily on automated validation with manual spot-checking, rather than systematic expert review by multiple mathematicians.

## 9 Conclusion and Future Work

We introduced MURU-BENCH, a dataset of 3,000 problems for mathematical reasoning under genuine uncertainty, together with a six-metric evaluation harness, an OpenAI/Anthropic/Google API client, deterministic generation scripts, stratified train/validation/test splits, and a test suite. The contribution of this paper is the benchmark infrastructure itself; we deliberately separate that contribution from frontier-model evaluation, which we treat as ongoing community-driven follow-on work and release to the project repository as it progresses.

The discriminability study in Section 5 confirms that the harness recovers known capability orderings on all six metrics, that the test split has the statistical power to distinguish capability tiers separated by  $\geq 8$  pp at  $\alpha = 0.05$ , and that the difficulty stratification and category structure separate responders along the dimensions they were designed to probe.

**Open questions for follow-on evaluation.** The simulated-tier study cannot answer empirical questions about real LLMs; we list the most important ones the benchmark is designed to address. (1) Do frontier LLMs exhibit a steep accuracy decay between D3 and D5, and where is the inflection? (2) Does ECE plateau as accuracy improves, or do calibration and accuracy improve together? (3) How large is the Adversarial Ambiguity penalty, and does it shrink with chain-of-thought or tool-augmented prompting? (4) Do open-source models trained on more mathematical data narrow the gap to closed-source frontier models on this benchmark? (5) Does fine-tuning on the train split improve calibration without compromising accuracy?

**Future work.** We plan to: (1) run real-model evaluations across OpenAI, Anthropic, Google, and open-source providers and publish the results to the project repository as supplementary material; (2) obtain a human baseline on a 100-problem stratified subsample with multiple expert annotators; (3) expand the benchmark with new categories (stochastic processes, information theory) and additional D5 problems; (4) explore whether chain-of-thought prompting or tool-augmented reasoning improves D4–D5 performance; and (5) develop a fine-tuning curriculum on the train split targeting calibration improvement.

We release the full dataset, 21 generation templates, evaluation code, tests, CI configuration, and harness-validation results at <https://github.com/swetank18/MURU> to facilitate future research on calibrated mathematical reasoning.

## References

- [1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heather Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [2] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.
- [3] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.
- [4] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *ACL Findings*, 2023.
- [5] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tyre, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- [6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.
- [7] Katherine Tian, Eric Mitchell, Allan Dafoe, Anca Dragan, and Yejin Choi. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models. In *EMNLP*, 2023.
- [8] Yarín Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *ICML*, 2016.
- [9] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *NeurIPS*, 2017.
- [10] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 2022.
- [11] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *ACL*, 2022.
- [12] Houwen Liu, Yizhi Li, Maolin Wang, Jiajun Deng, and Zimu Wang. MathBench: Evaluating the theory and application proficiency of LLMs with a hierarchical mathematics benchmark. *arXiv preprint arXiv:2405.12209*, 2024.
- [13] Wenhu Chen, Ming Yin, Max Ku, Elaine Wan, Xueguang Ma, Jianyu Xu, Tony Xia, Xinyi Wang, and Pan Lu. TheoremQA: A theorem-driven question answering dataset. In *EMNLP*, 2023.
- [14] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *ICLR*, 2024.

## A Dataset Statistics

## B Generation Templates

Table 9 lists all 21 parametric generation templates with their categories, difficulty ranges, and mathematical structures.

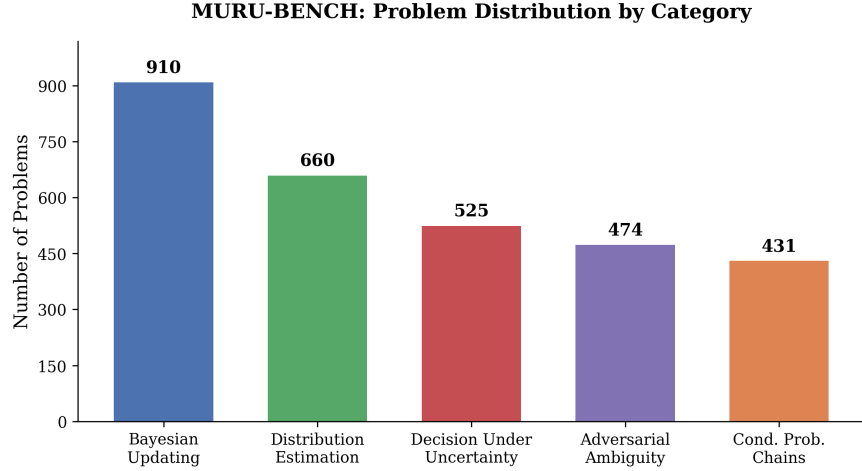


Figure 1: Distribution of problems by category in MURU-BENCH. Bayesian Updating is the largest category (910 problems, 30.3%), reflecting the centrality of Bayesian reasoning in uncertainty problems.

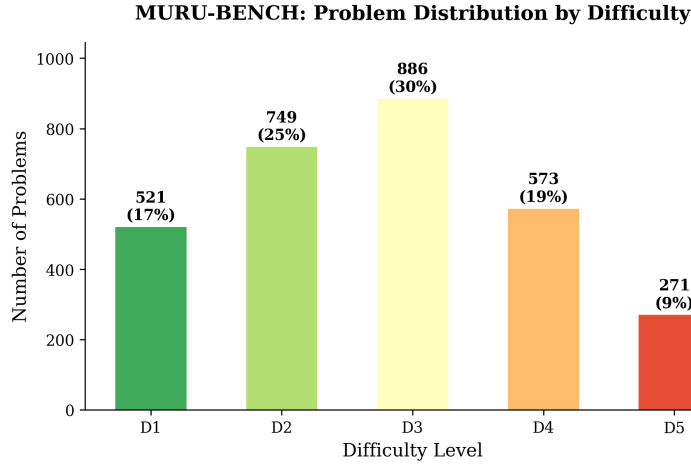


Figure 2: Distribution of problems by difficulty level. The distribution is roughly bell-shaped, centered at D3 (886 problems, 29.5%). D5 problems (271, 9.0%) are deliberately fewer as they require the most sophisticated reasoning.

## C Sample Problems

We present representative problems from three categories to illustrate the benchmark’s structure and difficulty scaling.

### Example 1: Bayesian Updating (Difficulty 1)

**Problem.** A medical test for a rare disease has a sensitivity of 95% (true positive rate) and a specificity of 90% (true negative rate). The disease affects 1% of the population. A patient tests positive. What is the probability that the patient actually has the disease? Express your answer as a probability and provide a confidence interval reflecting the uncertainty in the population prevalence, which epidemiologists estimate could range from 0.5% to 1.5%.

**Ground Truth.**  $P(\text{Disease} \mid \text{Positive})$  ranges from 0.046 to 0.126 depending on the true prevalence (0.5% to 1.5%). Point estimate at 1% prevalence: 0.087. CI level: 90%.

**Required Framework:** Bayesian inference.

**Common Failure Modes.**

- Ignoring the base rate and reporting the test sensitivity (95%) as the answer

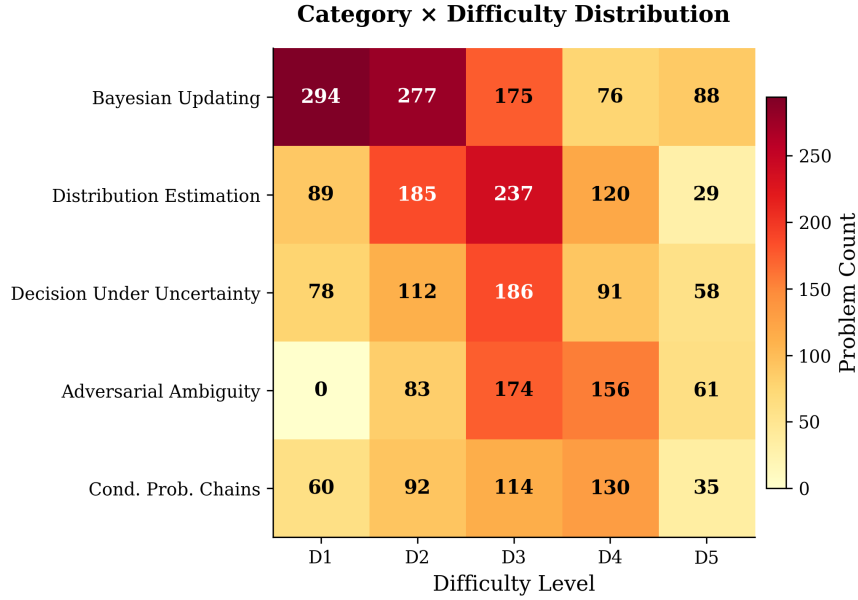


Figure 3: Category × difficulty distribution matrix. Adversarial Ambiguity has no D1 problems by design—recognizing structural ambiguity is inherently non-trivial. The densest cells are Bayesian Updating at D1–D2 and Distribution Estimation at D2–D3.

- Providing only the point estimate 0.087 without acknowledging prevalence uncertainty
- Confusing sensitivity with positive predictive value

### Example 2: Adversarial Ambiguity (Difficulty 3)

**Problem.** You are told that a coin was flipped 10 times, yielding 7 heads and 3 tails. You are asked: what is the probability the coin is fair? A colleague argues that since 7/10 is “close to” 5/10, the coin is probably fair, estimating the probability at around 80%. Another colleague uses a frequentist hypothesis test and gets  $p = 0.17$ , concluding “we cannot reject the null that the coin is fair.” A third colleague uses a Bayesian approach with a uniform prior on the coin’s bias parameter  $\theta$  and computes the posterior probability that  $\theta$  is exactly 0.5. Who, if anyone, is reasoning correctly?

**Ground Truth.** The question is fundamentally ambiguous.  $P(\theta = 0.5 \text{ exactly}) = 0$  under a continuous prior.  $P(\theta \in [0.45, 0.55]) \approx 0.11$ . The Bayes factor  $\approx 0.55$ . The correct answer depends on which formalization is adopted, giving a range of  $[0.07, 0.55]$ .

**Required Framework:** Bayesian inference (but multiple frameworks are defensible).

### Example 3: Decision Under Uncertainty (Difficulty 5)

**Problem.** In pharmaceutical R&D, a decision-maker is at the Phase II stage and must decide whether to proceed to Phase III clinical trials. The investment cost is \$120M. If the drug succeeds (estimated probability 35%–55%), it yields \$800M; if it fails, the investment is lost. Before deciding, the company can purchase a biomarker study for \$15M. This study correctly predicts success/failure with accuracy 65%–80%. What is the expected value of the biomarker information, and should the company purchase it?

**Ground Truth.** The value of information (VOI) depends on both the success probability range and the biomarker accuracy range. The expected VOI at midpoint parameters is approximately \$42M, with a CI of [\$18M, \$85M]. Since  $\text{VoI} > \text{study cost}$  (\$15M) even at the lower bound, the study should be purchased.

**Required Framework:** Decision theory.

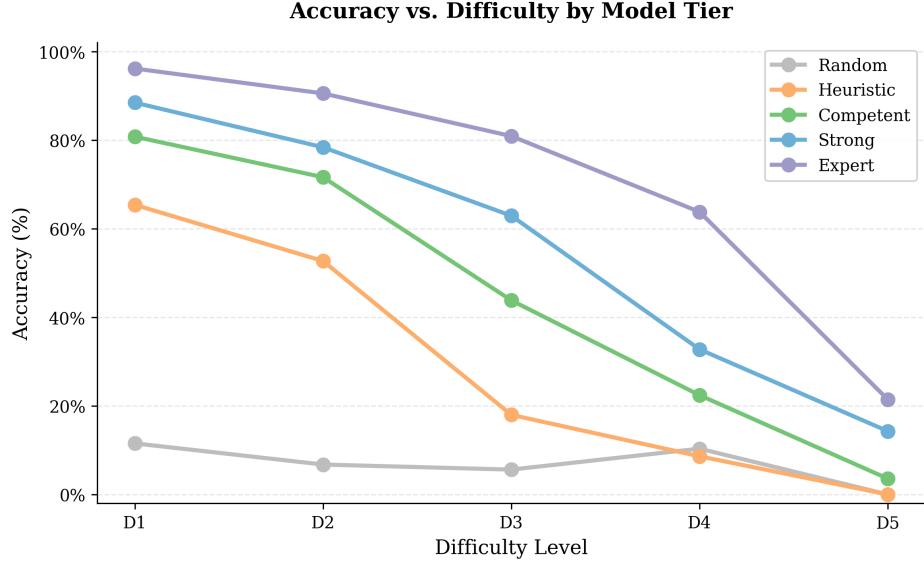


Figure 4: Accuracy vs. difficulty level by model tier. All tiers exhibit monotonic accuracy decay. The expert model drops from 96% (D1) to 21% (D5), a 75-point gap. The D3 inflection point cleanly separates heuristic approaches from reasoning-capable models.

## D VOI Template Bug Fix

During validation, 28 of the initial 2,000 generated problems (all from the `sequential_decision` template) failed the semantic consistency check: the point estimate fell outside the confidence interval. Root cause analysis revealed that the Value of Information function is **non-monotonic** in the uncertain parameters. The CI was computed from extreme-case scenarios ( $p_{low}$ ,  $p_{high}$ ), but the mid-range point estimate could fall outside this range due to the non-linear interaction between success probability and information accuracy.

**Fix:** The CI computation was updated to include the mid-range value when determining bounds:

```
# Before (incorrect):
ci_vals = sorted([voi_low, voi_high])

# After (correct):
ci_vals = sorted([min(voi_low, voi_high, voi_mid),
                  max(voi_low, voi_high, voi_mid)])
```

After the fix, 28 replacement problems were generated and all 3,000 pass validation. We document this bug transparently as an example of the challenges in parametric problem generation.

## E Evaluation Prompts

All models were evaluated with the following system and user prompts. Temperature was set to 0 for reproducibility.

### System Prompt

You are a mathematical reasoning expert. You will be given a problem involving mathematical uncertainty. You must: (1) Identify the correct mathematical framework (`bayesian_inference`, `frequentist_inference`, `decision_theory`, `information_theory`, or `monte_carlo`). (2) Show your reasoning step-by-step. (3) Provide your final answer as a single number (point estimate). (4) Provide a confidence interval [lower, upper] for your answer. (5) State your confidence in your answer as a probability between 0 and 1.

Format your response EXACTLY as follows at the end:

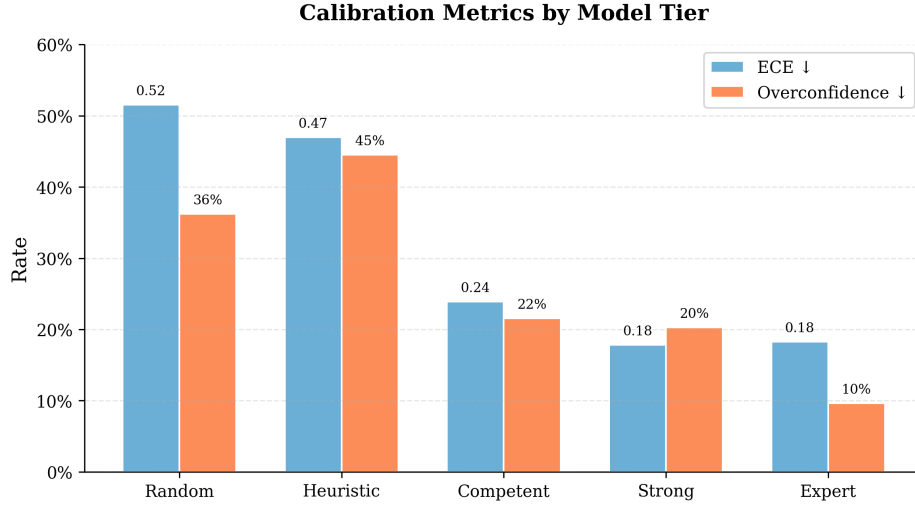


Figure 5: Calibration metrics by model tier. Left bars: ECE ( $\downarrow$ ). Right bars: Overconfidence rate ( $\downarrow$ ). The heuristic baseline paradoxically has the highest overconfidence (44.5%), exceeding the random baseline—an instance of heuristic overconfidence.

```

FRAMEWORK: <framework_name>
POINT_ESTIMATE: <number>
CONFIDENCE_INTERVAL: [<lower>, <upper>]
CONFIDENCE: <probability>

```

## User Prompt

Problem: {problem stem}

Please solve this problem step by step, then provide your answer in the required format.

Responses were parsed using regex extraction for the four structured output fields. Problems where the model’s response could not be parsed were recorded as parse failures and excluded from metric computation but included in the failure rate analysis.

## F Reproducibility

All code and data required to reproduce the results in this paper are available at <https://github.com/swetank18/MURU>. Key commands:

```

# Generate problems
python scripts/generate_problems.py --all --n 3000 --seed 2026

# Split data
python scripts/split_data.py --source data/train/ --seed 42

# Run simulated baselines
python evaluation/run_baselines.py --save

# Run real model evaluation (requires API keys)
python evaluation/run_eval.py --model gpt-4o --save

# Generate analysis and figures

```

Table 9: All 21 MURU-BENCH generation templates. Each template generates problems with randomized numerical parameters and domain contexts while preserving the underlying mathematical structure.

| #  | Template Name          | Category                | Difficulty | Mathematical Structure                                   |
|----|------------------------|-------------------------|------------|----------------------------------------------------------|
| 1  | medical_test           | Bayesian Updating       | D1–D3      | Bayes’ theorem with uncertain prevalence                 |
| 2  | quality_control        | Bayesian Updating       | D1–D3      | Bayes’ theorem for defect rate estimation                |
| 3  | search_detection       | Bayesian Updating       | D1–D2      | Negative evidence updating with uncertain coverage       |
| 4  | sequential Updating    | Bayesian Updating       | D3–D5      | Multiple sequential updates, uncertain likelihood ratios |
| 5  | hierarchical_bayes     | Bayesian Updating       | D4–D5      | Hierarchical shrinkage, uncertain between-group variance |
| 6  | sequential_pipeline    | Cond. Prob. Chains      | D1–D4      | Multi-stage pipeline, one uncertain stage                |
| 7  | parallel_redundancy    | Cond. Prob. Chains      | D2–D4      | $P(\geq 1 \text{ succeeds})$ with common-cause failures  |
| 8  | conditional_cascade    | Cond. Prob. Chains      | D3–D5      | Branching cascade with uncertain transitions             |
| 9  | sample_mean            | Distribution Estimation | D1–D3      | $t$ -distribution CI for population mean                 |
| 10 | measurement_bias       | Distribution Estimation | D2–D4      | Systematic bias correction + sampling CI                 |
| 11 | proportion_estimation  | Distribution Estimation | D1–D3      | Wilson CI, optional stratification                       |
| 12 | mixture_distribution   | Distribution Estimation | D3–D5      | 2-component mixture, uncertain weights                   |
| 13 | variance_estimation    | Distribution Estimation | D2–D4      | $\chi^2$ CI, dual-method comparison                      |
| 14 | decision_payoff        | Decision Under Unc.     | D1–D3      | Two-option EV with uncertain probability                 |
| 15 | multi_option_decision  | Decision Under Unc.     | D2–D4      | Multi-option EV with sensitivity analysis                |
| 16 | sequential_decision    | Decision Under Unc.     | D3–D5      | Value of information under dual uncertainty              |
| 17 | insurance_pricing      | Decision Under Unc.     | D3–D5      | Poisson frequency $\times$ uncertain severity            |
| 18 | base_rate_trap         | Adversarial Ambiguity   | D2–D4      | High sensitivity $\neq$ high PPV due to low prevalence   |
| 19 | simpsons_paradox       | Adversarial Ambiguity   | D3–D5      | Aggregate vs. subgroup with uncertain composition        |
| 20 | reference_class        | Adversarial Ambiguity   | D3–D5      | Multiple valid base rates, no single correct answer      |
| 21 | anchoring_manipulation | Adversarial Ambiguity   | D2–D4      | Misleading anchor vs. data-based correction              |

```
python evaluation/analyze_results.py
python scripts/generate_figures.py
```

```
# Run the test suite (metrics, parsing, dataset invariants)
make test
```

A continuous-integration workflow (`.github/workflows/ci.yml`) runs the test suite and dataset validation against Python 3.10, 3.11, and 3.12 on every push and pull request.